

Race Condition Lab

1. 实验环境

本次实验大家可以选择在Lab1的环境中完成，也可以选择其他的Ubuntu环境中做。其中Task1-3推荐大家在Lab1的环境中做，不容易出问题。

如果是用服务器的同学，在完成task1-3后，务必记得恢复 `/etc/passwd` 文件。

2.Task 1-3

即原本的seedlab的Race Condition Vulnerability Lab，大家参照 `task1-3` 文件夹下的 `task1-3.pdf` 实验文档完成。

3.Task 4

这里我们有一个程序 `task4.c`，其逻辑是创建一个或多个线程，这些线程都会循环对一个全局共享变量 `shared_counter` 进行加一操作，直到 `shared_counter` 的值到达上限 `maxcounter`。即如下代码：

```
1 void* worker_func(void* args) {
2     int id = *((int*)args);
3
4     int my_counter = 0;
5     while (shared_counter < maxcounter) {
6         ++my_counter;
7         ++shared_counter;
8     }
9
10    pthread_exit((void*)my_counter);
11 }
```

可以看到每个线程有一个自己的变量 `my_counter`，每次对 `shared_counter` 加一时也会对 `my_counter` 加一。

3.1 统计信息

`task4` 文件夹下有两个python文件 `test_currency.py` 和 `test_time.py` 用来统计程序的运行结果。

- `test_currency.py` 分别测试创建1、2、4、8和16个线程时（`maxcounter` 为10000000），线程运行结束后 `sum(my_counter)/maxcounter` 的值，`sum(my_counter)` 为每个线程的 `my_counter` 的和。
- `test_time.py` 分别测试创建1、2、4、8和16个线程时（`maxcounter` 为10000000），程序运行的总时间。

跑一下 `test_currency.py` 和 `test_time.py` 描述你观察到的现象。

Q: 当存在多个线程时，可以看到程序结束后 `sum(my_counter)` 不等于 `maxcounter`，为什么？

3.2 mutex

利用mutex用来保证 `sum(my_counter)=maxcounter`，使得最后 `test_currency.py` 的结果为100%。大家可以使用 `pthread` 库自带的 `pthread_mutex` 锁，相应API及使用方式大家可以自行搜索。

用 `test_currency.py` 和 `test_time.py` 进行测试，描述相应结果。

3.3 spinlock

利用spinlock用来保证 `sum(my_counter)=maxcounter`，使得最后 `test_currency.py` 的结果为100%。

第一步，先使用 `pthread` 库自带的 `pthread_spinlock`，相应API及使用方式大家可以自行搜索。用 `test_currency.py` 和 `test_time.py` 进行测试，描述相应结果。

第二步，要求大家使用C语言自行实现spinlock，然后用自己实现的spinlock进行测试。要求 `test_time.py` 的结果和使用 `pthread_spinlock` 时相近。

4.Task 5

这里我们有一个程序 `task5.c`，核心是函数 `vuln` 和 `check`，`vuln` 将一个文件的全部内容存入一个char数组 `buf` 中，`buf` 的大小为256字节，因此为了防止buffer overflow，在写入之前使用 `check` 检查了文件的大小，若文件大小大于255字节，则其拒绝写入。以下是部分代码，其余逻辑参见源代码。

```
1 void showflag() { printf("flag{race_condition_succeed!}\n"); }
2
3 void vuln(char* file) {
4     char buf[256];
5
6     int number;
7     int index = 0;
8
9     int fd = open(file, O_RDONLY);
10    if (fd == -1) {
11        perror("open file failed!!");
12        return;
13    }
14    while (1) {
15        number = read(fd, buf + index, 128);
16        if (number <= 0) {
17            break;
18        }
19        index += number;
20    }
21 }
22
23 void check(char* file) {
24     struct stat tmp;
25     stat(file, &tmp);
26     if (tmp.st_size > 255) {
27         puts("file size is too large!!");
28         exit(0);
29     }
30 }
```

```
31  
32 int main(){  
33     /* check then vuln */  
34 }
```

在检查完文件到将文件内容写入 `buf` 之间会存在一个时间窗口，要求大家利用这个时间窗口，想办法让程序调用 `showflag` 函数，该函数原本并不会被调用。

程序编译需要关闭栈保护：

```
1 | gcc task5.c -o <prog> -fno-stack-protector
```

同时可以选择关闭地址随机化：

```
1 | sudo sysctl -w kernel.randomize_va_space=0
```